

MATEMÁTICA COMPUTACIONAL APLICADA

Aula 14

Estudo de Árvore

O grafo mais simples que existe: conexo, sem ciclos, e cheio de estrutura escondida.

raiz

folha

altura

BST



Nesta aula



Árvore como um grafo especial

Conexo e sem ciclos — um caso particular do que vimos na Aula 12.



Terminologia

Raiz, folha, altura, profundidade, subárvore.



Árvore binária e árvore binária de busca

No máximo 2 filhos por nó, e a regra que viabiliza busca rápida.



Percursos

Pré-ordem, em-ordem e pós-ordem.



O que é uma árvore

Uma árvore é um grafo conexo e sem ciclos — os mesmos conceitos da Aula 12, só que combinados de um jeito específico.

Propriedades importantes:

- Uma árvore com n vértices tem exatamente $n - 1$ arestas.
- Existe um único caminho entre quaisquer dois vértices.
- Removendo qualquer aresta, a árvore se desconecta em duas partes.

CONTANDO ARESTAS DE UMA ÁRVORE



$$\text{arestas} = \text{vértices} - 1$$

Como uma árvore com n vértices tem sempre $n - 1$ arestas, uma árvore com 10 vértices tem 9 arestas — sem precisar desenhá-la.



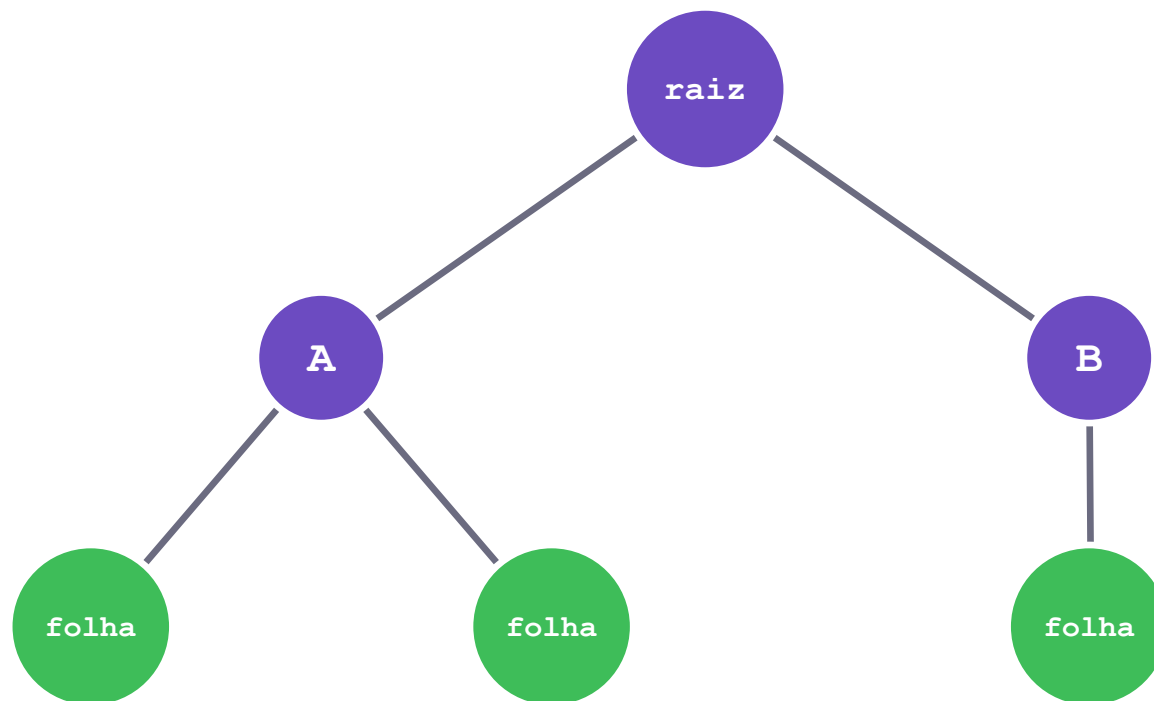
Terminologia

Numa árvore enraizada, escolhemos um vértice como raiz e organizamos o resto em hierarquia:

Termo	Significado
Raiz	vértice do topo, sem pai
Pai / filho	relação entre um nó e os nós ligados diretamente abaixo dele
Folha	nó sem filhos
Profundidade de um nó	nº de arestas da raiz até ele
Altura da árvore	maior profundidade entre todas as folhas
Subárvore	uma árvore "dentro" da árvore, enraizada em qualquer nó

Profundidade e altura

A profundidade de um nó é a distância (em arestas) até a raiz. A altura da árvore é a maior profundidade entre as folhas.

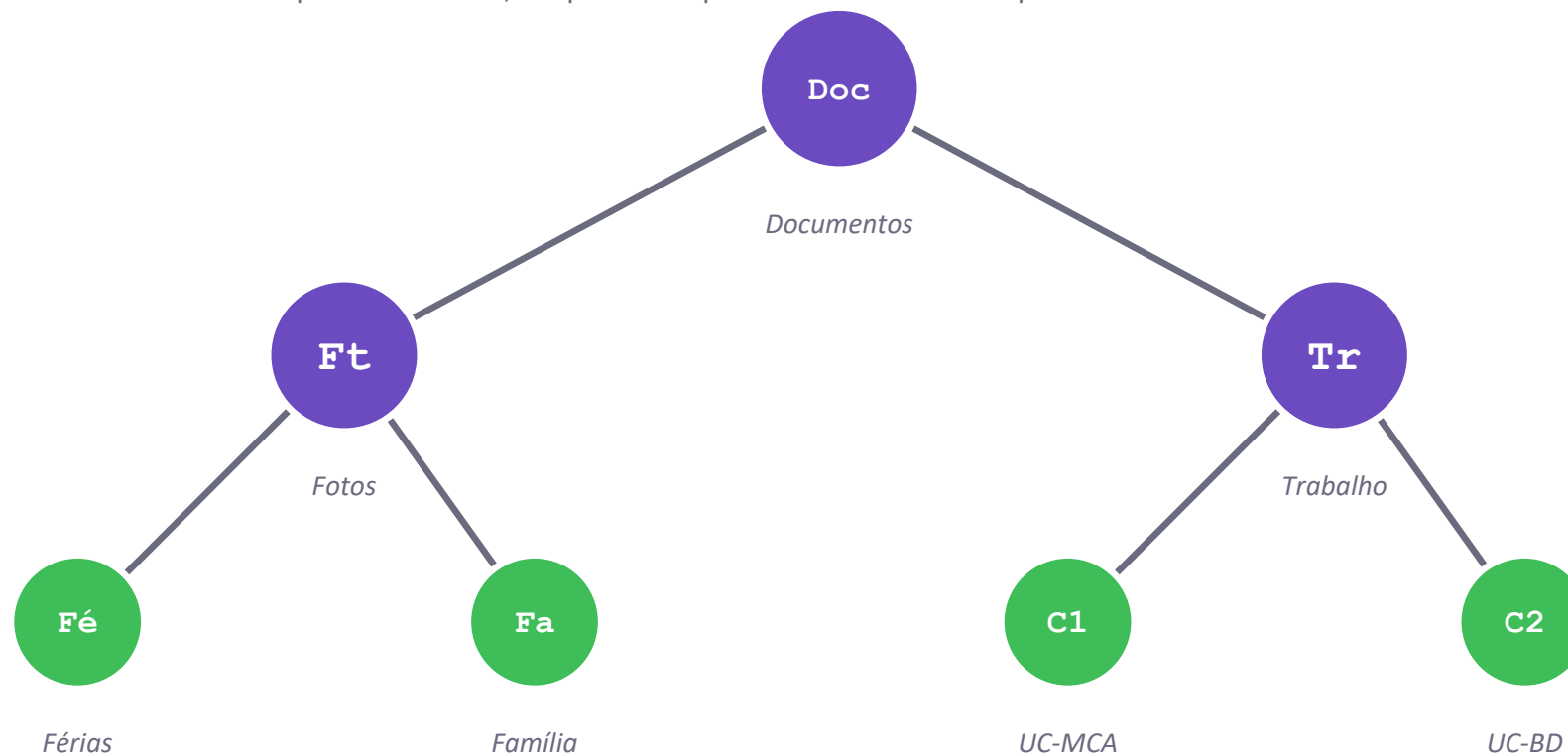


Aqui a raiz tem profundidade 0, A e B têm profundidade 1, e as folhas têm profundidade 2 — logo a altura da árvore é 2.



Pastas de arquivos

Documentos é a raiz; Fotos e Trabalho têm profundidade 1; as quatro subpastas mais baixas têm profundidade 2.



Repare que "Trabalho" tem 2 filhos, mas nada impede uma pasta real de ter 3, 4 ou mais — diferente da árvore binária da próxima seção.



Árvore binária e árvore binária de busca

Árvore binária: cada nó tem no máximo 2 filhos (esquerdo e direito).

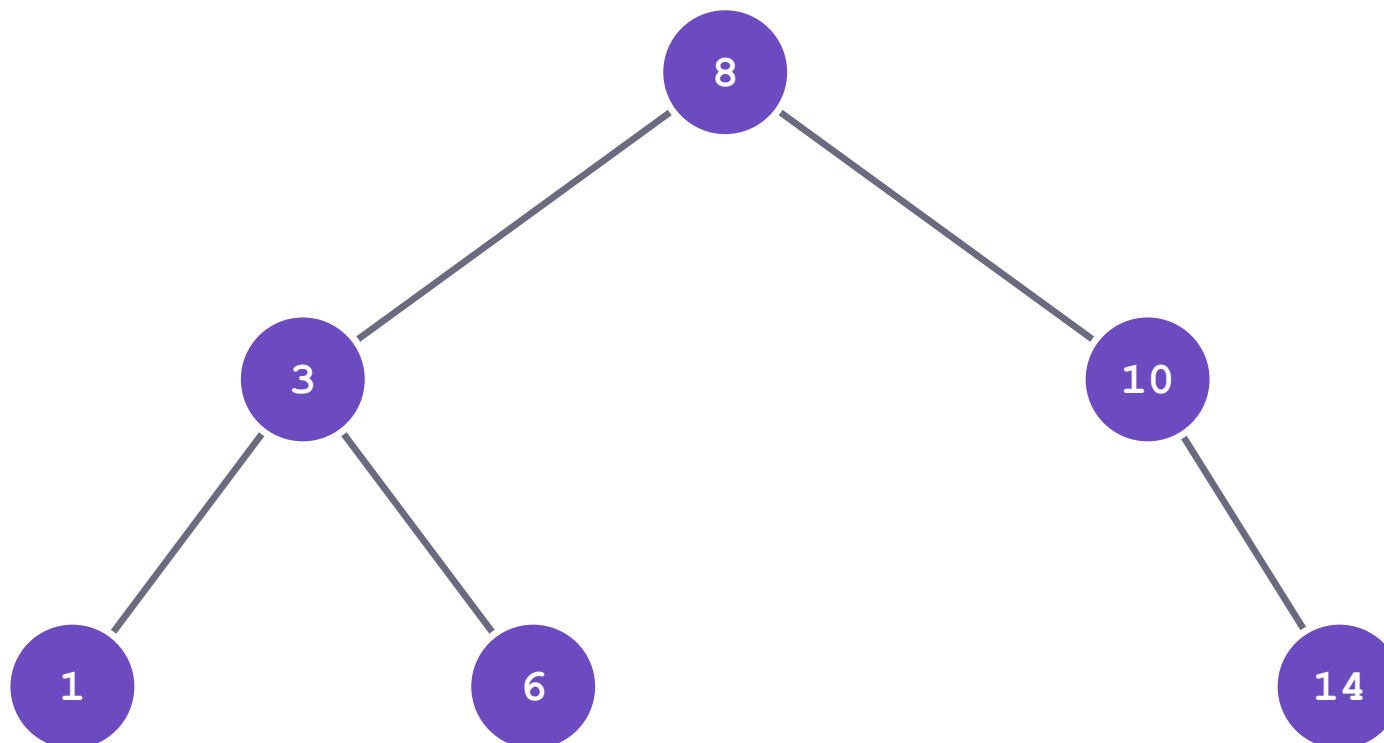
Árvore binária de busca (BST): para todo nó, os valores da subárvore esquerda são menores, e os da subárvore direita são maiores.

Essa regra é o que permite buscar um valor em $O(\log n)$ passos, em vez de percorrer todos os n nós.

Toda BST é uma árvore binária, mas nem toda árvore binária é uma BST — a diferença é a regra de ordenação.

Uma árvore binária de busca

Raiz 8, com filho esquerdo 3 (filhos 1 e 6) e filho direito 10 (filho direito 14):



Regra da BST: filhos à esquerda são sempre menores; filhos à direita, sempre maiores.

Percorrendo uma árvore



Pré-ordem

Visita o nó, depois a esquerda, depois a direita.

ex.: copiar a estrutura da árvore



Em-ordem

Visita a esquerda, depois o nó, depois a direita.

ex.: numa BST, dá a ordem crescente



Pós-ordem

Visita a esquerda, a direita, depois o nó.

ex.: apagar a árvore (filhos antes do pai)



Árvore binária em código

Um nó com valor, filho esquerdo e filho direito — e uma função recursiva para cada percurso.

Definindo o nó

Cada nó guarda um valor e referências para até dois filhos.

No.cs



```
public class No {  
    public int Valor;  
    public No Esquerda, Direita;  
    public No(int valor,  
        No esq = null, No dir = null) {  
        Valor = valor;  
        Esquerda = esq; Direita = dir;  
    }  
}
```

no.py



```
class No:  
    def __init__(self, valor,  
        esquerda=None, direita=None):  
        self.valor = valor  
        self.esquerda = esquerda  
        self.direita = direita
```

Percurso em-ordem

Esquerda, depois o nó, depois a direita — numa BST isso visita os valores em ordem crescente.

No.cs



```
static List<int> EmOrdem(No no,
    List<int> resultado = null) {
    resultado ??= new List<int>();
    if (no == null) return resultado;
    EmOrdem(no.Esquerda, resultado);
    resultado.Add(no.Valor);
    EmOrdem(no.Direita, resultado);
    return resultado;
}

var arvore = new No(8,
    new No(3, new No(1), new No(6)),
    new No(10, null, new No(14)));
var resultado = EmOrdem(arvore);
```

arvore.py



```
def em_ordem(no, resultado=None):
    if resultado is None:
        resultado = []
    if no is None:
        return resultado
    em_ordem(no.esquerda, resultado)
    resultado.append(no.valor)
    em_ordem(no.direita, resultado)
    return resultado

arvore = No(8, No(3, No(1), No(6)),
    No(10, None, No(14)))
```

`print(em_ordem(arvore))` `# [1, 3, 6, 8, 10, 14]` *-> ordem crescente!*



Em-ordem confirma que é uma BST

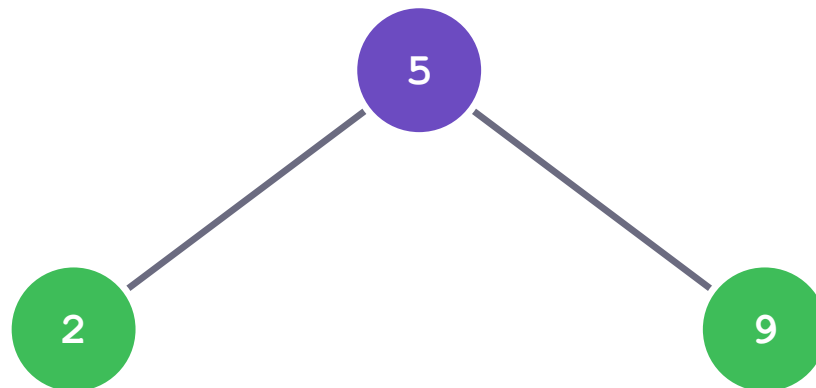
Se a travessia em-ordem de uma árvore binária NÃO sai em ordem crescente, ela NÃO satisfaz a propriedade de BST.

Em outras palavras: em-ordem é um teste de sanidade de uma linha para checar se a estrutura está correta.

COMPARANDO AS TRÊS ORDENS

Pré, em e pós-ordem lado a lado

Árvore com raiz 5, filho esquerdo 2 e filho direito 9 (ambos folhas):



Pré-ordem

[5, 2, 9]

raiz primeiro

Em-ordem

[2, 5, 9]

raiz no meio

Pós-ordem

[2, 9, 5]

raiz por último

Mesmos elementos nas três — só a posição da raiz muda. Use essa regra para conferir o Exercício 3.

Onde árvores aparecem



Sistemas de arquivos

Pastas dentro de pastas — o exemplo que já vimos nesta aula.



DOM do HTML

Elementos aninhados formam uma árvore que o navegador percorre para renderizar a página.



Índices de banco de dados

B-trees organizam registros para buscas rápidas em milhões de linhas.



Árvores de decisão

Um dos modelos mais usados em aprendizado de máquina.



Hora de praticar

Quatro exercícios: contagem de arestas, profundidade e altura, percursos numa BST e um desafio de implementação.



EXERCÍCIO 1

Contando arestas

Uma árvore tem 15 vértices. Quantas arestas ela possui? Justifique usando a propriedade vista na aula.

arestas = _____

Justificativa: _____



GABARITO · EXERCÍCIO 1

Contando arestas

$$\text{arestas} = \text{vértices} - 1 = 15 - 1 = 14$$

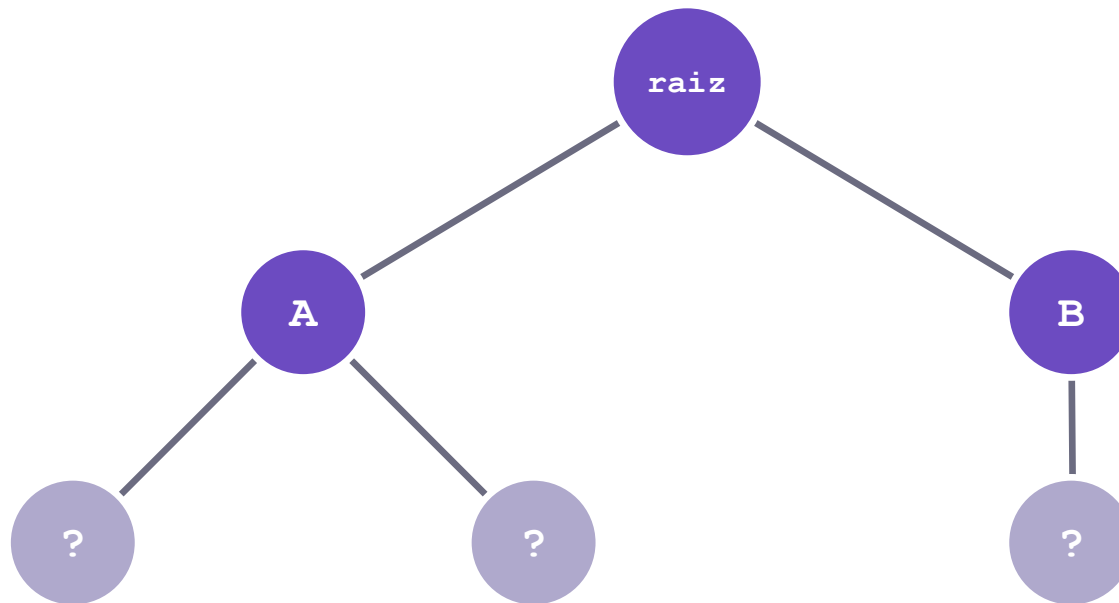
Toda árvore com n vértices tem exatamente $n - 1$ arestas: não há como fechar um ciclo sem violar a definição de árvore.



EXERCÍCIO 2

Profundidade e altura

Para a árvore do diagrama de terminologia (raiz, A, B, três folhas), identifique a profundidade de cada nó e a altura da árvore.

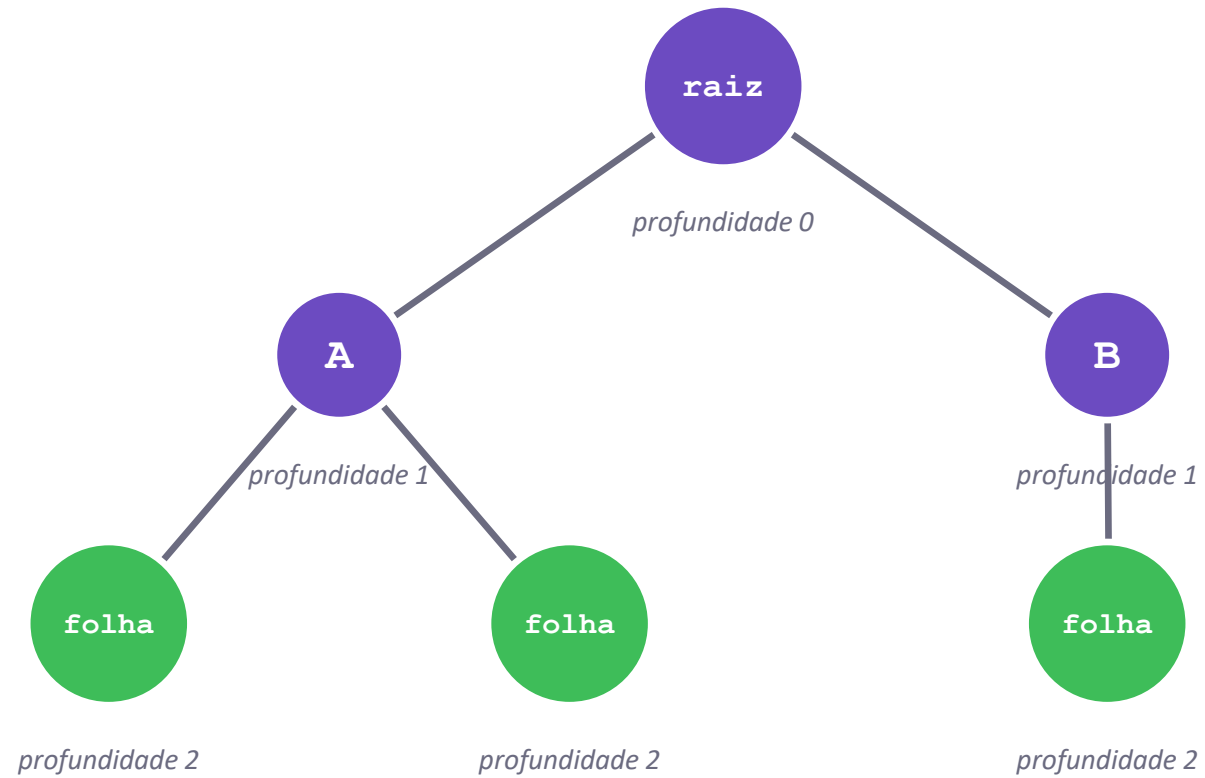


Profundidade: raiz=___ A=___ B=___ folhas=___ Altura=___



GABARITO · EXERCÍCIO 2

Profundidade e altura



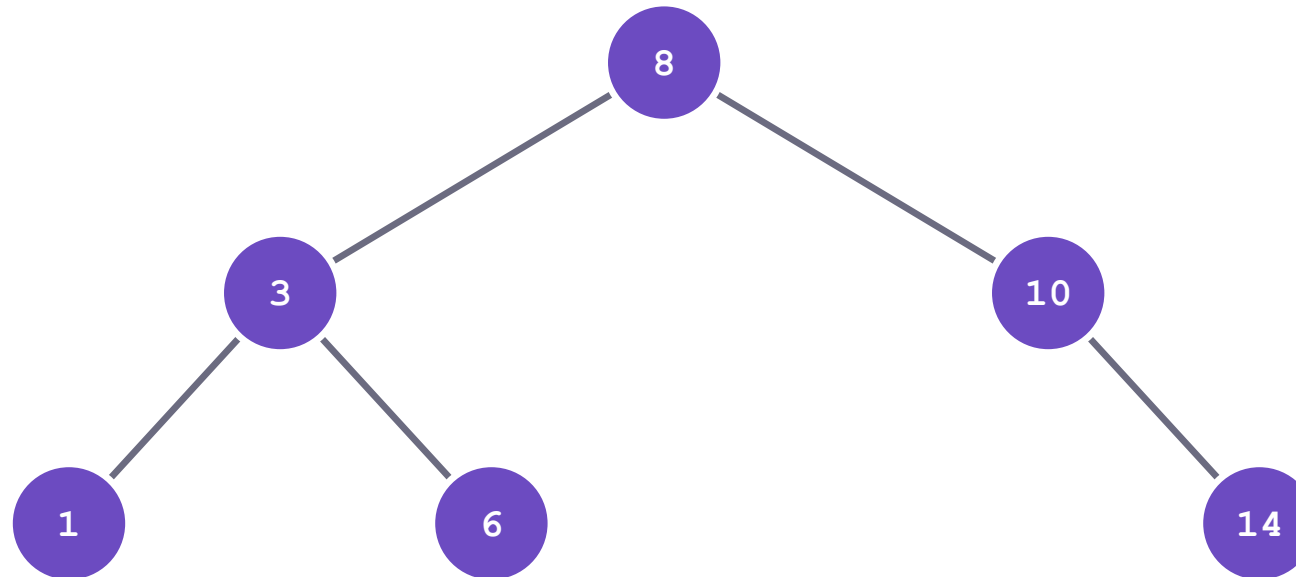
Altura da árvore = 2 (maior profundidade entre as folhas)



EXERCÍCIO 3

Pré-ordem e pós-ordem numa BST

Dada a BST com raiz 8, filho esquerdo 3 (com filhos 1 e 6) e filho direito 10 (com filho direito 14), escreva a ordem de visita em pré-ordem e em pós-ordem (a em-ordem já está na aula).



Pré-ordem: [_____]

Pós-ordem: [_____]



GABARITO · EXERCÍCIO 3

Pré-ordem e pós-ordem numa BST

Pré-ordem = [8, 3, 1, 6, 10, 14]

Pós-ordem = [1, 6, 3, 14, 10, 8]

Pré-ordem visita 8, depois toda a subárvore esquerda (3,1,6), depois toda a direita (10,14). Pós-ordem visita as duas subárvores antes de visitar 8 por último.



EXERCÍCIO 4 · DESAFIO

Implementando os percursos

Implemente `pre_ordem(no)` e `pos_ordem(no)` em Python, seguindo o mesmo padrão de `em_ordem` da aula. Teste com a árvore de exemplo.



`pre_pos_ordem.py`

```
def pre_ordem(no, resultado=None): _____
```

(mesmo padrão do `em_ordem` — só muda a posição do `append`)

```
def pos_ordem(no, resultado=None): _____
```

(o `append` vem depois das duas chamadas recursivas)

teste: `em_ordem`, `pre_ordem` e `pos_ordem` devem bater com o Exercício 3



GABARITO · EXERCÍCIO 4

Implementando os percursos

pre_ordem.py



```
def pre_ordem(no, resultado=None):  
    if resultado is None:  
        resultado = []  
    if no is None:  
        return resultado  
    resultado.append(no.valor)  
    pre_ordem(no.esquerda, resultado)  
    pre_ordem(no.direita, resultado)  
    return resultado
```

pos_ordem.py



```
def pos_ordem(no, resultado=None):  
    if resultado is None:  
        resultado = []  
    if no is None:  
        return resultado  
    pos_ordem(no.esquerda, resultado)  
    pos_ordem(no.direita, resultado)  
    resultado.append(no.valor)  
    return resultado
```

Testado na árvore 8/3/10/1/6/14: pre_ordem → [8,3,1,6,10,14] pos_ordem → [1,6,3,14,10,8]

PARA IR ALÉM

Referências

Livros — teoria



ROSEN, K. H.

Matemática Discreta e suas Aplicações. 7. ed.
AMGH/McGraw-Hill — cap. Árvores.



CORMEN, T. H. et al.

Algoritmos: Teoria e Prática (CLRS). Elsevier —
cap. Árvores binárias de busca.



GOODRICH, M. T.; TAMASSIA, R.

Estruturas de Dados e Algoritmos em Python.
Bookman — árvores e percursos.

Documentação e vídeos



Python — recursão

[Funções que chamam a si mesmas](#) — documentação oficial.



Vídeos

[Árvores binárias de busca explicadas com animações](#)

[Percurso pré-ordem, em-ordem e pós-ordem passo a passo](#)



SÍNTESE

O grafo mais organizado que existe

**Árvore é grafo sem excesso:
só as arestas essenciais para conectar tudo.**

Guarde raiz, folha, profundidade e altura — e os três percursos (pré, em, pós) que vêm com eles.